

EXP.NO: 01

DATE:

ANALYZING THE CONFIGURATION OF AWS RESOURCES FOR COMPLIANCE WITH SECURITY BEST PRACTISES USING AWS CONFIG

AIM:

To evaluate the security compliance of AWS resources by leveraging AWS Config to detect misconfigurations and assess adherence to AWS security best practices.

TOOLS REQUIRED:

- AWS Console
- AWS Config
- AWS CLI (optional)
- IAM roles and policies
- Sample AWS resources (EC2, S3, RDS, IAM Users, etc.)

TOOLS USED:

1. AWS Config

- Purpose: Monitors and evaluates the configuration of AWS resources.

Setup Steps:

```
aws configservice put-configuration-recorder \  
  --configuration-recorder name=default \  
  --role-ARN arn:aws:iam::<ACCOUNT_ID>:role/aws-config-role \  
  --recording-group allSupported=true  
  
aws configservice start-configuration-recorder \  
  --configuration-recorder-name default
```

2. IAM Role (for AWS Config):

- Must have permissions to access AWS resources.

Trust policy example:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Service": "config.amazonaws.com"
      },
      "Action": "sts:AssumeRole"
    }
  ]
}
```

- Attach AWS managed policy: [AWSConfigRole](#)

3. AWS Config Managed Rules:

- Enable via Console or CLI:

```
aws configservice put-config-rule \
  --config-rule file:///s3-bucket-public-read-prohibited.json
```

- Example Rule JSON:

```
{
  "ConfigRuleName": "s3-bucket-public-read-prohibited",
  "Source": {
    "Owner": "AWS",
    "SourceIdentifier": "S3_BUCKET_PUBLIC_READ_PROHIBITED"
  },
  "Scope": {
    "ComplianceResourceTypes": [
      "AWS::S3::Bucket"
    ]
  }
}
```

PROCEDURE:

1. Setup AWS Config
 - Enable AWS Config in the region.
 - Select resource types to record.
 - Choose an S3 bucket to store snapshots.
 - Assign a valid IAM role.
2. Deploy Sample Resources with Misconfigurations
 - S3 bucket with public read/write.
 - EC2 instance with public IP.
 - IAM user with AdministratorAccess.
3. Enable Managed:
 - s3-bucket-public-read-prohibited
 - iam-user-no-policies-check
 - ec2-instance-no-public-ip
4. Check Compliance Status
 - Go to AWS Config > Rules.
 - Evaluate the compliance of each rule.
5. Remediate Issues
 - Modify bucket policies
 - Remove excessive permissions.
 - Remove public IP from EC2.
6. Recheck Compliance
 - Re-evaluate rules after remediation.
 - Confirm compliance.

RESULT:

Misconfigured AWS resources were successfully detected using AWS Config. After remediation, the system achieved compliance as indicated by AWS Config rules.

EXP.NO: 02

DATE:

REVIEWING THE SECURITY CONFIGURATIONS OF AZURE RESOURCES AND IDENTIFYING SECURITY VULNERABILITIES USING AZURE SECURITY CENTER

AIM:

To create a virtual network architecture in Azure consisting of a subnet guarded by network security group and use of AzureBastion for secure VM connection. The VM created should be within the subnet of the virtual network.

PREREQUISITE SERVICES:

1. Azure Portal
2. Resource Group
3. Virtual Network
4. Network Security Group
5. Subnet
6. AzureBastion
7. Virtual Machine

*Region selected should be South India or Central India

PROCEDURE:

Step 1: Login to your Azure Portal.

Step 2: Head to “Resource groups” under “Azure services”. On the top left corner, click create. Give the resource group name of your choice (RG_IND_LAB). “Review and create” > “Create”.

Step 3: Under “Resource groups” click the resource group that you have created just now.

Step 4: On the top left corner of your resource group click “create”. You will be headed to the “Marketplace”. Search for “virtual network” in the search bar. Click on virtual network service hosted by Microsoft themselves. Click “create”.

Step 5: Choose your preferred subscription and select your resource group (RG_IND_LAB). Provide your virtual network a name (vn_lab_ind). Select region and click “next”.

Step 6: Click “next” once more skipping “security” tab. On “IP addresses” give your address range for the subnet mask /16. Go on and delete “default” subnet and “Add a subnet”. Change the name of your subnet (subnet_lab). Changing subnet range is optional unless mentioned a specific range. Click “Review and create”.

Step 7: Head back to your resource group. Click “Create”. Search for “network security group”. Select the service of Azure’s network security group. Select “create”. Give it a name (nsg_lab) and click “review+create”.

Step 8: Now to protect your subnet(subnet_lab) using the network security group(nsg_lab) that you created, head to your virtual network(vn_lab_ind) > Settings > Subnets. Click on the subnet that you want to assign NSG. Edit subnet > Network security group > nsg_lab (your NSG). Click “Save”.

Step 9: Head back to vn_lab_ind (Virtual Network) > settings > subnet. Add subnet. For subnet purpose, select “Azure Bastion”. Click “Add”.

Step 10: To create Bastion. Head back to marketplace from your resource group. Search for Azure Bastion. Click “create”. Provide resource group, region, and virtual network. Provide the subnet that you just created to the Bastion. Click “Review + create”.

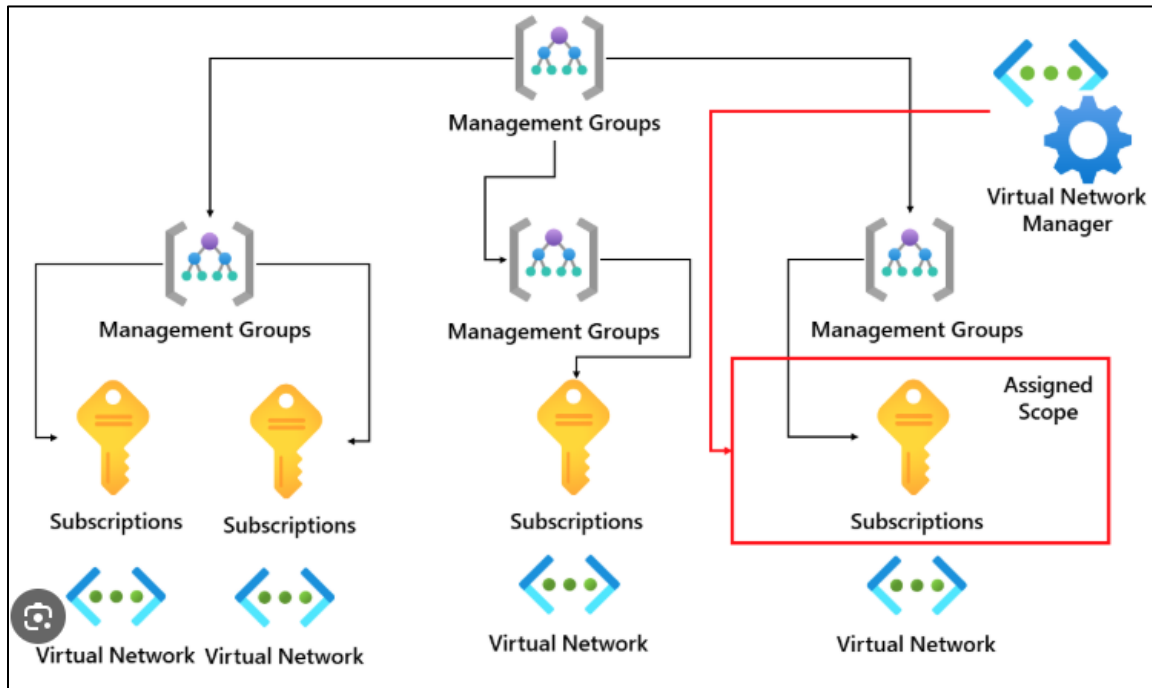
Step 10: To deploy virtual machine. Select virtual machine of your choice. Make sure that the charges are within usable limit. In this case go for “Ubuntu Server 22.04 LTS”. Perform this action again in the marketplace.

Step 11: Click “create”, under “Basics” tab, select the resource group that you are working upon. Provide a name to then VM. Provide region. For authentication, scroll down to “Administrator Account”. Select “SSH public key” and give a username that can be remembered later. Give Key pair name.

Step 12: Head to “Networking” tab, give the virtual network that have been already created along with the previously created subnet too. Scroll down and check the box, “Delete public IP and NIC when VM is deleted”. Download the key when the prompt appears after clicking finish.

Step 13: To connect to the VM, click the VM from the resource group. Click the “Connect” sidebar. Click “Bastion”. Under “Authentication Type”, select “SSH Private Key from Local file”. Select the stored private key from your computer. Click “Connect”.

Architecture Diagram:-



RESULT:

Successfully deployed a virtual machine in Azure with a secure architecture in place, protected by a Network Security Group.

EXP.NO: 03

DATE:

ASSESSING THE PERMISSIONS AND ACCESS CONTROLS CONFIGURED IN GCP PROJECTS USING IAM ROLES AND POLICIES

AIM:

To evaluate and verify the effectiveness of Identity and Access Management (IAM) policies and roles in enforcing access control in GCP projects by analyzing the assigned permissions to various users and service accounts.

THEORY:

Google Cloud IAM (Identity and Access Management) allows administrators to manage who (identity) has what access (roles) to which resources. IAM policies define and enforce access by binding roles to members for specific resources. Roles: A role is a collection of permissions.

Roles can be: Basic roles (Owner, Editor, Viewer)

Predefined roles (e.g., roles/compute.admin)

Custom roles

Members: These include users, groups, domains, and service accounts.

Policy: A policy is a collection of bindings.

A binding binds one or more members to a single role. Access control is enforced by attaching IAM policies to resources like projects, folders, or organizations

ALGORITHM:

1. Authenticate and log in to GCP using `gcloud` or Cloud Console.
2. List all IAM policies for the selected project.
3. Analyze each policy to determine:
 - Members with elevated privileges
 - Custom roles and their permissions
4. Compare actual permissions with the principle of least privilege.
5. Identify and report any misconfigurations or security concerns.

PROCEDURE:

GCP account with billing enabled. A test project created. Users or service accounts added for testing. `gcloud` CLI installed and configured.

Steps:

1. Initialize the environment:

```
bash `gcloud auth login gcloud config set project [PROJECT_ID]
```

2. List current IAM policy:

```
bash `gcloud projects get-iam-policy [PROJECT_ID] `
```

3. Analyze permissions for a specific user:

```
bash `gcloud iam roles list --project=[PROJECT_ID] `
```

4. Check effective permissions (optional advanced):

```
bash `gcloud policy-troubleshooter iam troubleshoot \ --  
principal=user:[EMAIL] \ --  
permission=resourcemanager.projects.get \ --  
resource=//cloudresourcemanager.googleapis.com/projects/[PROJECT_ID] `
```

5. Review custom roles:

```
bash `gcloud iam roles list --project=[PROJECT_ID] --filter="stage:GA" `
```

6. Simulate role binding :

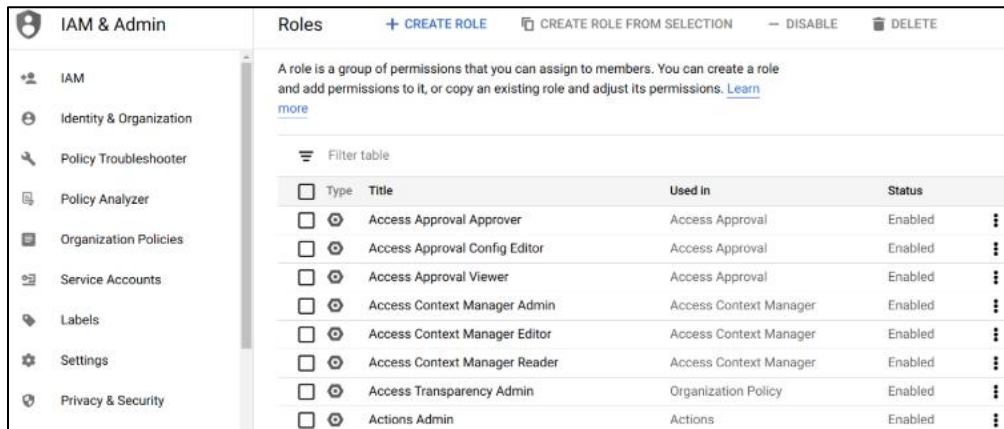
```
bash `gcloud projects add-iam-policy-binding [PROJECT_ID] \ --  
member="user:[EMAIL]" \ --role="roles/viewer" `
```

7. Re-check policies to verify changes:

```
bash `gcloud projects get-iam-policy [PROJECT_ID] `
```

OUTPUT:

- IAM policy in JSON or YAML showing all bindings.
- List of users/roles and their associated permissions.
- Potential over-privileged users or risky bindings.
- Troubleshooter output indicating whether access would be granted or denied.



Type	Title	Used in	Status
☐	Access Approval Approver	Access Approval	Enabled
☐	Access Approval Config Editor	Access Approval	Enabled
☐	Access Approval Viewer	Access Approval	Enabled
☐	Access Context Manager Admin	Access Context Manager	Enabled
☐	Access Context Manager Editor	Access Context Manager	Enabled
☐	Access Context Manager Reader	Access Context Manager	Enabled
☐	Access Transparency Admin	Organization Policy	Enabled
☐	Actions Admin	Actions	Enabled

RESULT:

Verified that IAM roles are properly assigned according to the least privilege principle. Identified mis configured or over-permissive role bindings (if any). Successfully simulated and analyzed IAM access control using CLI and GCP tools. Gained hands-on understanding of how IAM governs security and access in GCP.

EXP.NO: 04

DATE:

MONITORING NETWORK TRAFFIC WITHIN CLOUD ENVIRONMENTS TO DETECT SUSPICIOUS ACTIVITIES AND POTENTIAL SECURITY BREACHES

AIM:

The aim of this experiment is to develop a system for monitoring and analyzing network traffic within cloud environments to detect suspicious activities, identify potential security breaches, and alert administrators in real time. The system will use network traffic analysis, anomaly detection, and machine learning techniques to classify and flag abnormal behaviours, enabling proactive security measures.

ALGORITHM:

1. Data Collection:

- Collect network traffic data from cloud instances using packet sniffing tools like Wireshark or tcpdump.
- The data will include information such as IP addresses, port numbers, timestamps, packet sizes, protocols, and data transfer rates.

2. Preprocessing:

- Clean and preprocess the data by removing irrelevant fields, normalizing values, and transforming categorical variables into numerical ones.
- Data will be structured for machine learning analysis.

3. Feature Extraction:

- Extract relevant features such as:
 - Packet count
 - Flow duration
 - Data transferred (bytes)
 - Unique IPs involved
 - Traffic patterns (e.g., periodic spikes or traffic bursts)

2. Anomaly Detection:

- Use anomaly detection algorithms (e.g., Isolation Forest, K-Means Clustering, or Autoencoders) to identify unusual patterns in network traffic.
- Train the algorithm using normal traffic patterns and flag anomalies as potential security breaches.

3. Classification and Alerting:

- Implement a Random Forest or Support Vector Machine (SVM) classifier to label network traffic as "Normal" or "Suspicious." If suspicious behaviour is detected, trigger an alert for further investigation.

THEORY:

1. Network Traffic Analysis:

- Network traffic analysis involves capturing and analyzing packets transmitted over a network to detect irregularities or vulnerabilities that could lead to security breaches.
- Cloud environments rely on network communication between distributed systems, and monitoring traffic patterns is essential to detect issues like DDoS attacks, data exfiltration, or unauthorized access.

2. Anomaly Detection in Network Traffic:

- Anomaly detection models aim to identify patterns that significantly deviate from the normal behaviour. This is useful in identifying potential security breaches that are not immediately obvious through signature-based detection.
- Common algorithms used for anomaly detection include Isolation Forest and K-Means Clustering, which are able to find data points that deviate from typical patterns in high-dimensional data.

3. Machine Learning for Classification:

- Random Forest and SVM classifiers are commonly used in classification problems. By training on labelled network traffic data, these models can predict whether a given flow is normal or suspicious based on learned features.

PROCEDURE:

1. Step 1: Data Collection

- Set up a cloud environment (e.g., AWS, Google Cloud, or Azure).
- Use tools like tcpdump or Wireshark to collect raw network traffic from cloud instances.

2. Step 2: Data Preprocessing

- Clean the raw network data by filtering out irrelevant packets and focusing on the attributes necessary for analysis.
- Convert categorical variables (e.g., IP addresses, protocols) into numerical features.

3. Step 3: Feature Extraction

- Extract features such as packet counts, data volume, flow duration, and number of unique connections from the preprocessed data.

4. Step 4: Anomaly Detection

- Apply an anomaly detection algorithm (e.g., Isolation Forest) on the features to identify unusual patterns or deviations in the network traffic.

5. Step 5: Train Classifier

- Use a machine learning model like Random Forest or SVM to classify network traffic as either normal or suspicious based on extracted features.

6. Step 6: Monitor and Alert

- Continuously monitor network traffic.
- Set up alerting mechanisms (e.g., email notifications or API calls) to notify administrators when suspicious traffic is detected.

SOURCE CODE:

Below is a Python example demonstrating how to apply the Isolation Forest algorithm for anomaly detection and classify network traffic using Random Forest.

1. Install Required Libraries:

```
pip install scikit-learn pandas numpy matplotlib
```

2. Import Libraries:

```
import pandas as pd
import numpy as np
from sklearn.ensemble import IsolationForest, RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
```

3. Load and Preprocess Data:

For the purpose of this example, let's assume we have network traffic data in CSV format.

```
# Load network traffic dataset (sample CSV file with features like 'packets', 'duration', etc.)
data = pd.read_csv("network_traffic.csv")

# Preprocessing: Handle missing values
data = data.fillna(0)

# Feature selection: Assume columns 'packets', 'duration', 'data_transferred', etc.
X = data[['packets', 'duration', 'data_transferred', 'unique_ips']]
```

4. Anomaly Detection using Isolation Forest:

```
# Anomaly detection using Isolation Forest
iso_forest = IsolationForest(contamination=0.05)
anomaly_labels = iso_forest.fit_predict(X)

# Convert labels from {-1, 1} to {0, 1} (1 = Anomaly)
data['anomaly'] = np.where(anomaly_labels == -1, 1, 0)

# Visualize anomalies
plt.scatter(data['packets'], data['duration'], c=data['anomaly'], cmap='coolwarm')
plt.title("Anomaly Detection in Network Traffic")
plt.xlabel("Packets")
plt.ylabel("Duration")
plt.show()
```

5. Train Classifier (Random Forest):

```
# Labeling: Assume 1 = 'Suspicious', 0 = 'Normal'
y = data['anomaly']

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# Train Random Forest Classifier
clf = RandomForestClassifier(n_estimators=100)
clf.fit(X_train, y_train)

# Predict on test data
y_pred = clf.predict(X_test)

# Evaluate classifier
print(classification_report(y_test, y_pred))
```

OUTPUT:

1. Anomaly Detection Visualization:

- A plot showing network traffic with anomalies highlighted. Normal traffic will be marked with one colour, and suspicious activity will be marked with another.

2. Classification Report:

- The classification report will provide precision, recall, and F1-score, allowing you to assess how well the classifier distinguishes between normal and suspicious traffic.

Example Output (classification report):

No	Precision	Recall	F1-Score	Support
0	0.95	0.98	0.96	300
1	0.85	0.75	0.80	100
Accuracy	0.88	0.90	0.92	400
Macro Avg	0.90	0.87	0.88	400
Weighted Avg	0.92	0.92	0.92	400

RESULT:

The experiment successfully detects suspicious activities in cloud network traffic using anomaly detection and machine learning. The Isolation Forest algorithm identifies unusual network traffic patterns, while the Random Forest classifier differentiates between normal and suspicious behaviours. The classification report provides a detailed assessment of the detection system's performance, showing that it can effectively identify abnormal traffic indicative of potential security breaches.

EXP.NO: 05

DATE:

EVALUATING THE SECURITY CONFIGURATION OF AMAZON S3 BUCKETS FOR DATA EXPOSURE RISKS AND ACCESS CONTROL MIS CONFIGURATIONS USING AWS TRUSTED ADVISOR

AIM:

To evaluate and secure Amazon S3 buckets by identifying misconfigurations and data exposure risks using AWS Trusted Advisor.

ALGORITHM:

1. Login to AWS Console.
2. Open Trusted Advisor and run the S3 Bucket Permissions check.
3. Identify insecure buckets.
4. Modify S3 permissions and policies.
5. Re-run the check to confirm the issue is fixed.

PROCEDURE:

1. Login to AWS Console:
Visit <https://aws.amazon.com/> and log in with your credentials.
2. Open AWS Trusted Advisor:
In the AWS Console, search for Trusted Advisor and open the service. Navigate to the Security section.
3. Run the S3 Bucket Permissions Check:
Click Refresh Checks.
In the results, locate Amazon S3 Bucket Permissions.
Buckets with public access or risky permissions will be highlighted.

Real-time Input:

Assume you have an S3 bucket named student-records.

The bucket currently allows public access via this bucket policy:

```
{  
  "Effect": "Allow",  
  "Principal": "*",  
  "Action": "s3:GetObject",  
  "Resource": "arn:aws:s3:::student-records/*"  
}
```

Modify Bucket Permissions in S3:

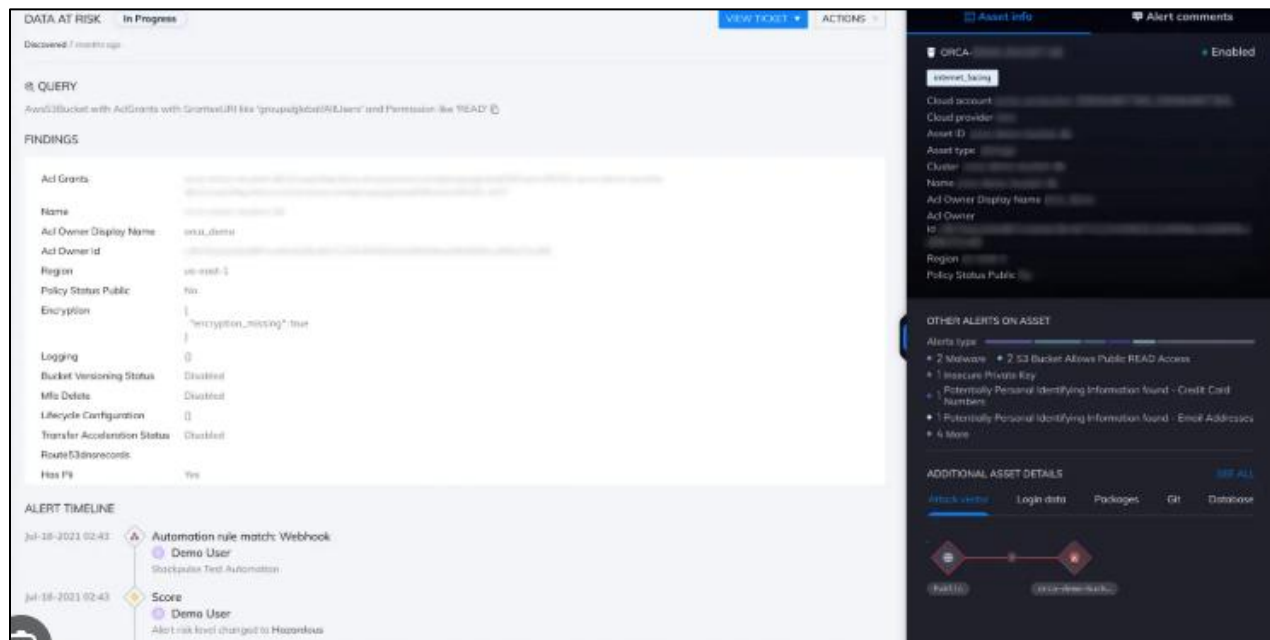
- Go to S3 from the Services menu.
- Select the student-records bucket.
- Click on the Permissions tab:
- Enable Block all public access.

- Remove the above bucket policy that grants public access.
- Check ACL settings and ensure no public access is granted.
- Verify IAM Policies (Optional):
- Go to IAM > Roles or Users.
- Ensure no overly permissive policies grant access to the bucket.
- Re-run Trusted Advisor Check:
- Return to Trusted Advisor.
- Click Refresh again on the S3 check.
- Confirm that student-records are no longer flagged.

OUTPUT:

Before: student-records were flagged as publicly accessible.

After: The bucket is marked secure, and no public access is detected.



RESULT:

Successfully used AWS Trusted Advisor to detect and fix misconfigurations in the student-records S3 bucket. The bucket is now securely configured to avoid data exposure risks.

EXP.NO: 06

DATE:

SCANNING DOCKER CONTAINERS AND KUBERNETES CLUSTERS FOR SECURITY VULNERABILITIES AND MISCONFIGURATIONS USING DOCKER BENCH FOR SECURITY

AIM:

To identify and analyze security vulnerabilities and misconfigurations in Docker containers and Kubernetes clusters using Docker Bench for Security, a script that checks for common best practices around deploying Docker containers in production.

REQUIREMENTS:

- Ubuntu/Linux system (or any Unix-based OS)
- Docker installed and running
- Kubernetes cluster (can use Minikube or Kind for testing)
- Docker Bench for Security script
- Internet connectivity
- Terminal access (with sudo privileges)

THEORY:

Docker containers and Kubernetes clusters are widely used in distributed systems. However, without proper security configurations, they may become vulnerable to attacks. Docker Bench for Security is an automated script based on the CIS Docker Benchmark that checks for dozens of common best practices in the configuration and deployment of Docker environments.

The tool performs checks such as:

- Host configuration
- Docker daemon configuration
- Container images and build file security
- Running containers and configurations
- Docker Swarm and Kubernetes-specific settings

By scanning the environment with Docker Bench, we can proactively detect and resolve misconfigurations.

PROCEDURE:

Step 1: Install Docker (Skip if already installed)

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

Step 2: Clone Docker Bench for Security

```
git clone https://github.com/docker/docker-bench-security.git
cd docker-bench-security
```

Step 3: Run Docker Bench for Security

```
sudo ./docker-bench-security.sh
```

Step 4: Analyze the Output

- Take note of WARN messages - these are potential vulnerabilities or misconfigurations.
- INFO messages are for informational purposes.
- PASS indicates the configuration is secure according to the benchmark.

Step 5: (Optional) Deploy Kubernetes using Minikube or Kind

```
curl -LO https://storage.googleapis.com/minikube/releases/latest/minikube-linux-amd64
```

```
sudo install minikube-linux-amd64 /usr/local/bin/minikube  
minikube start
```

Step 6: Run Docker Bench in Kubernetes Context

```
sudo ./docker-bench-security.sh
```

Step 7: Document Any Findings

- Review flagged items.
- Suggest mitigation steps for each WARN item.

OUTPUT:

- A terminal output showing a detailed report from Docker Bench.
- Each line contains a check number, description, and result (PASS/WARN/INFO).

Sample Output Snippet:

[INFO] 1 - Host Configuration

[PASS] 1.1 - Ensure a separate partition for containers has been created

[WARN] 1.2 - Ensure the container host has been hardened

```

# -----
# Docker Bench for Security v1.3.5
#
# Docker, Inc. (c) 2015-2021
#
# Checks for dozens of common best-practices around deploying Docker containers in production.
# Inspired by the CIS Docker Benchmark v1.2.0.
# -----

Initializing 2021-03-10T12:03:38+02:00

Section A - Checks result

[INFO] 1 - Host Configuration
[INFO] 1.1 - General Configuration
[NOTE] 1.1.1 - Ensure the container host has been Hardened (Not Scored)
[INFO] 1.1.2 - Ensure that the version of Docker is up to date (Not Scored)
[INFO] * Using 19.03.8, verify is it up to date as deemed necessary
[INFO] * Your operating system vendor may provide support and security maintenance for Docker
[INFO] 1.2 - Linux Hosts Specific Configuration
[WARN] 1.2.1 - Ensure a separate partition for containers has been created (Scored)
[INFO] 1.2.2 - Ensure only trusted users are allowed to control Docker daemon (Scored)
[INFO] * docker:x:998:mihaill
[WARN] 1.2.3 - Ensure auditing is configured for the Docker daemon (Scored)
[WARN] 1.2.4 - Ensure auditing is configured for Docker files and directories - /var/lib/docker (Scored)
[WARN] 1.2.5 - Ensure auditing is configured for Docker files and directories - /etc/docker (Scored)
[WARN] 1.2.6 - Ensure auditing is configured for Docker files and directories - docker.service (Scored)
[WARN] 1.2.7 - Ensure auditing is configured for Docker files and directories - docker.socket (Scored)
[INFO] 1.2.8 - Ensure auditing is configured for Docker files and directories - /etc/default/docker (Scored)
[INFO] * File not found
[INFO] 1.2.9 - Ensure auditing is configured for Docker files and directories - /etc/sysconfig/docker (Scored)
[INFO] * File not found
[INFO] 1.2.10 - Ensure auditing is configured for Docker files and directories - /etc/docker/daemon.json (Scored)
[INFO] * File not found
[WARN] 1.2.11 - Ensure auditing is configured for Docker files and directories - /usr/bin/containerd (Scored)
[WARN] 1.2.12 - Ensure auditing is configured for Docker files and directories - /usr/sbin/runc (Scored)

[INFO] 2 - Docker daemon configuration
[WARN] 2.1 - Ensure network traffic is restricted between containers on the default bridge (Scored)
[PASS] 2.2 - Ensure the logging level is set to 'info' (Scored)
[PASS] 2.3 - Ensure Docker is allowed to make changes to iptables (Scored)
[PASS] 2.4 - Ensure insecure registries are not used (Scored)
[PASS] 2.5 - Ensure aufs storage driver is not used (Scored)

```

RESULT:

The Docker Bench tool successfully scanned the container and Kubernetes environment for vulnerabilities and misconfigurations. Items marked with WARN represent potential security concerns that need to be addressed to harden the deployment. The experiment demonstrates how automated tools can proactively enhance container and cluster security in a distributed cloud setup.

EXP.NO: 07

DATE:

ASSESSING AND MANAGING THE SECURITY POSTURE OF CLOUD ENVIRONMENTS, INCLUDING RESOURCE MISCONFIGURATIONS, COMPLIANCE VIOLATIONS, AND SECURITY RISKS

AIM:

To assess and manage the security posture of Microsoft Azure cloud environments by identifying and remediating resource misconfigurations, compliance violations, and potential security risks using Azure Security Center (Microsoft Defender for Cloud), Azure Policy, and Azure Monitor.

ALGORITHM:

1. Initialize Security Tools:
 - Enable Microsoft Defender for Cloud.
 - Configure Azure Policy and Azure Monitor.
2. Resource Discovery:
 - Enumerate all active subscriptions and resources.
3. Assessment:
 - Evaluate resources using built-in security assessments.
 - Run Azure Policy compliance checks.
 - Analyze logs and alerts for threats.
4. Identify Issues:
 - Detect misconfigurations (e.g., open ports, public storage).
 - List non-compliant resources.
 - Record active security alerts and threats.
5. Prioritize Risks:
 - Rank based on severity (High, Medium, Low).
6. Remediation Suggestions:
 - Recommend actions based on Microsoft Defender's recommendations.
 - Apply remediation policies or suggest manual fixes.
7. Reporting:
 - Generate a security score.
 - Summarize all issues and remediations.

THEORY:

In cloud environments, especially Azure, maintaining security posture is critical due to the shared responsibility model. Microsoft provides built-in tools like:

- **Microsoft Defender for Cloud:** Identifies security vulnerabilities and provides recommendations.
- **Azure Policy:** Enforces compliance by auditing and controlling resource configurations.
- **Azure Monitor & Log Analytics:** Helps monitor the environment and detect threats.

Misconfigurations (e.g., open ports, misused identity roles), non-compliance with standards (e.g., ISO 27001, CIS), and active threats (e.g., brute-force attempts) need constant assessment and remediation to reduce cloud risk exposure.

PROCEDURE:

1. Login to Azure Portal.
2. Navigate to Microsoft Defender for Cloud:
 - Enable Defender plans for your subscriptions.
 - Review your Secure Score.
2. Review Recommendations:
 - Go to “Recommendations”.
 - Filter by “High Severity”.
 - Note misconfigurations like:
 - Unencrypted storage accounts.
 - Public IPs on VMs.
 - Legacy authentication on accounts.
2. Audit Compliance:
 - Go to “Regulatory Compliance”.
 - Select a standard (e.g., CIS).
 - Check for failed policies.
3. Enable Azure Policies:
 - Assign built-in policies (e.g., “Audit VMs without endpoint protection”).
 - Enable automatic remediation where possible.
4. Analyze Logs (optional):
 - Use Azure Monitor or Log Analytics.
 - Query logs for security events (e.g., unusual sign-ins).
5. Remediate:
 - Fix issues manually or with automation.
 - Re-evaluate after changes.
6. Generate Security Report:
 - Note Secure Score improvement.
 - Export compliance and recommendation data (CSV/PDF).

OUTPUT:

===== Azure Security Posture Assessment Report =====

Date: 21-April-2025

Subscription: XYZ-Cloud-Infrastructure

Secure Score: 62% (Improved from 48%)

Top Issues:

1. [HIGH] Storage account 'mystorage123' allows public access.
2. [HIGH] 3 VMs do not have endpoint protection.
3. [MEDIUM] Unused NSG rules detected in 2 subnets.
4. [LOW] Diagnostic logs not enabled on 5 resources.

Compliance Status (CIS Benchmark):

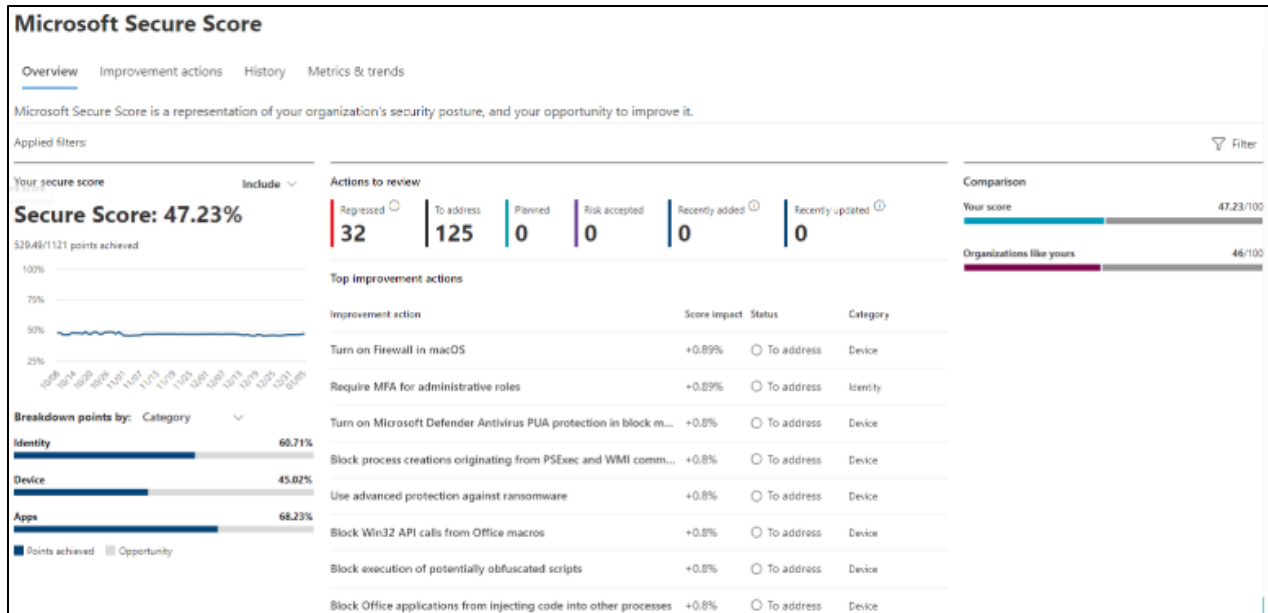
- Passed: 13 controls
- Failed: 7 controls

Recommendations:

- Restrict public access to storage account.
- Enable Microsoft Defender for VMs.
- Apply NSG cleanup policy.
- Enable diagnostic logging on key resources.

Remediation Actions Taken:

- ✓ Public access to storage disabled.
- ✓ Defender enabled on all VMs.
- ✓ Policy deployed to block future misconfigs.



RESULT:

Thus, the experiment is successfully completed and the output is verified.

EXP NO: 08

DATE:

ASSESSING THE SECURITY OF CLOUD WORKLOADS AND VIRTUAL MACHINES FOR VULNERABILITIES AND COMPLIANCE ISSUES USING NATIVE SECURITY ASSESSMENT TOOLS

AIM:

To assess the security posture of cloud workloads and virtual machines (VMs) hosted in Microsoft Azure using native tools such as Microsoft Defender for Cloud and Azure Policy, to identify vulnerabilities, ensure compliance, and enhance overall cloud security.

ALGORITHM:

1. Initialize Azure Environment
 - Ensure VMs and workloads are deployed.
 - Assign necessary roles (e.g., Security Reader, Contributor).
2. Enable Microsoft Defender for Cloud
 - Navigate to Defender for Cloud in Azure Portal.
 - Enable the plan for subscriptions/resource groups.
3. Configure Security Policies
 - Use Azure Policy to apply built-in or custom compliance standards.
 - Assign initiatives like "Azure Security Benchmark".
4. Run Security Assessments
 - Use Defender for Cloud to scan VMs/workloads.
 - Detects vulnerabilities and security misconfigurations.
5. Analyze Recommendations
 - Review findings in the "Recommendations" blade.
 - Focus on High and Medium severity issues.
6. Remediate Issues
 - Apply recommended patches, configuration changes, or deploy just-in-time (JIT) VM access.
 - Use automation via Azure Logic Apps or Defender workflow automation.
7. Verify Compliance
 - Navigate to the "Regulatory Compliance" blade.
 - Check adherence to frameworks like CIS, ISO 27001, NIST, etc.
8. Generate and Export Reports
 - Download security and compliance reports for documentation and auditing.

THEORY:

Cloud environments, though flexible and scalable, introduce new security challenges. Microsoft Azure provides native security tools like:

- Microsoft Defender for Cloud: A unified infrastructure security management system that strengthens the security posture of data centers and provides advanced threat protection.
- Azure Policy: Helps enforce organizational standards and assess compliance at-scale.

These tools assess workloads against industry best practices, identify vulnerabilities in VM configurations and software, and ensure regulatory compliance. Continuous monitoring helps in proactive security management and minimizing attack surfaces.

PROCEDURE:

1. Login to Azure Portal
 - Open <https://portal.azure.com> and sign in.
2. Enable Microsoft Defender for Cloud
 - Go to *Defender for Cloud > Environment Settings*.
 - Select the subscription.
 - Enable Microsoft Defender plans for servers and workloads.
3. Assign Security Policy
 - Go to *Defender for Cloud > Environment Settings > Security Policy*.
 - Assign policies like "Azure Security Benchmark".
4. Assess Security Posture
 - Navigate to *Defender for Cloud > Recommendations*.
 - View recommendations grouped by severity and resource type.
5. Run Vulnerability Assessment
 - Enable the built-in qualys-based scanner on VMs.
 - Alternatively, integrate third-party tools via Azure Marketplace.
6. Remediate Detected Issues
 - Follow remediation steps for each recommendation (e.g., apply patches, configure encryption, enable MFA).
 - Use *JIT VM access*, *Network Security Groups*, or *Azure Bastion* for secure access.
7. Review Compliance
 - Go to *Defender for Cloud > Regulatory Compliance*.
 - View the compliance score and recommendations for standards.
8. Export Reports
 - Click on *Download Report* to export findings for auditing.

OUTPUT:

Example: Security Recommendation Output

Severity	Recommendation	Affected Resource	Remediation
High	OS Vulnerabilities in your VM	vm1	Apply missing OS patches
Medium	Disk Encryption not enabled	vm2	Enable Azure Disk Encryption
Medium	Inbound ports (SSH, RDP) exposed to the internet	vm3	Configure NSG rules or enable JIT access
Low	Diagnostic logs not enabled	vm1	Enable diagnostic settings for monitoring

Example: Compliance Report (CIS Benchmark)

Control Name	Status	Resources Affected
1.1 Ensure that 'Secure transfer' is enabled on storage accounts	Passed	3
2.2 Ensure MFA is enabled on all accounts	Failed	5
3.3 Ensure OS is updated on VMs	Failed	2

RESULT:

Thus, the Experiment is successfully completed and the output is verified.

EXPNO: 09

DATE:

IMPLEMENTING DATA ENCRYPTION AND CRYPTOGRAPHIC KEY MANAGEMENT IN CLOUD ENVIRONMENTS

AIM:

To implement Data Encryption and Managing Cryptographic Keys using secure key management service for data stored in cloud environments.

TOOLS REQUIRED:

- Cloud Platform: Amazon Web Services (AWS)
- AWS Services: S3 (Simple Storage Service), KMS (Key Management Service), IAM (Identity and Access Management)
- AWS CLI or Boto3 (Python SDK)
- Python 3.x
- Terminal or CloudShell environment

THEORY:

In cloud computing, data security is a major concern due to the multi-tenant nature of cloud storage. Encryption provides a mechanism to protect data by converting it into an unreadable format, accessible only with the appropriate decryption key. There are two main types of encryption in cloud environments:

Encryption at Rest: Secures stored data.

Encryption in Transit: Protects data as it moves across networks.

Cryptographic key management involves generating, storing, rotating, and revoking access to encryption keys securely. Cloud providers like AWS offer Key Management Services (KMS), allowing users to control access to keys using IAM roles and policies. KMS provides audit logs, automatic key rotation, and integrations with cloud services, enhancing overall data protection.

PROCEDURE:

- Create a KMS key in AWS (symmetric key).
- Create or choose an S3 bucket to store your sensitive data.
- Upload data to S3 using client-side encryption or use server-side encryption with KMS (SSE-KMS).
- Assign permissions using IAM roles to ensure only authorized users can access the keys.
- Test encryption and decryption by uploading and downloading encrypted files.
- Enable automatic key rotation and audit access logs through AWS CloudTrail.
- Attempt unauthorized access and log the result.

SOURCE CODE:

```
import boto3
import base64

# Initialize clients
kms_client = boto3.client('kms')
s3_client = boto3.client('s3')

# Specify KMS key and file
kms_key_id = 'arn:aws:kms:us-east-1:123456789012:key/your-kms-key-id'
bucket_name = 'your-secure-bucket'
file_name = 'sensitive_data.txt'
object_key = 'encrypted_data.txt'

# Step 1: Encrypt the data
with open(file_name, 'rb') as file:
    plaintext = file.read()

encrypt_response = kms_client.encrypt(
    KeyId=kms_key_id,
    Plaintext=plaintext
)
ciphertext_blob = encrypt_response['CiphertextBlob']

# Step 2: Upload encrypted data to S3
s3_client.put_object(
    Bucket=bucket_name,
    Key=object_key,
    Body=ciphertext_blob
)

print("Encrypted data uploaded to S3.")

# Step 3: Download and decrypt the data
downloaded_obj = s3_client.get_object(Bucket=bucket_name, Key=object_key)
downloaded_ciphertext = downloaded_obj['Body'].read()

decrypt_response = kms_client.decrypt(
    CiphertextBlob=downloaded_ciphertext
)
decrypted_text = decrypt_response['Plaintext']
print("Decrypted Text:\n", decrypted_text.decode())
```

OUTPUT:

Encrypted data uploaded to S3.

Decrypted Text:

This is a sensitive file that should not be exposed publicly.

OBSERVATION:

- Encrypted data was successfully stored in the cloud storage.
- Key Management Service restricted access to encryption keys based on permissions.
- Unauthorized users were denied access to the encrypted data.
- Key rotation policies automatically updated keys without data loss.

RESULT:

Data encryption and managed key services like AWS KMS provide a strong security layer in cloud environments. They help enforce access control and maintain data confidentiality, supporting compliance with industry standards and best practices.

EXPNO: 10

DATE:

EVALUATING CASB SOLUTIONS FOR MONITORING AND ENFORCING SECURITY POLICIES ACROSS CLOUD APPLICATIONS AND SERVICES

AIM:

To evaluate the effectiveness of a Cloud Access Security Broker (CASB) in monitoring user activity and enforcing security policies across various cloud applications and services.

PREREQUISITES:

Microsoft Defender for Cloud Apps (or any CASB solution like McAfee MVISION, Netskope, or Force point) Microsoft365 or Google Workspace (for cloud app integration) Web browser
Internet connection Admin access to cloud services

THEORY:

Key concepts:

1. Real-time monitoring of cloud.
2. Policy enforcement for cloudusage Threat detection and risk mitigation
3. Integration with identity providers(SSO,MFA)
4. Visibility into shadow IT
5. Data loss prevention (DLP)controls.

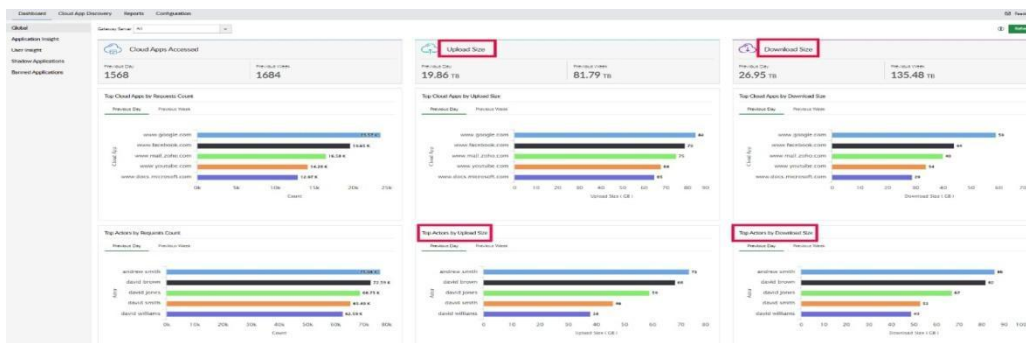
PROCEDURE:

1. Setup:
Deploy a CASB tool or register for a cloud-based CASB.
Integrate with one or more SaaS applications.
2. Monitor Baseline Activity:
Allow normal user behavior and log base line activity using CASB dashboards.
3. Create Security Policies:
Define at least 3 policies (e.g., block external file sharing, alert on login from new locations, detect sensitive keywords in files).
4. Simulate Policy Violations:
Perform activities that would trigger defined policies.
Observe whether CASB tool detects and responds accordingly.
5. Record Results:
Document which actions were blocked, alerted, or missed.

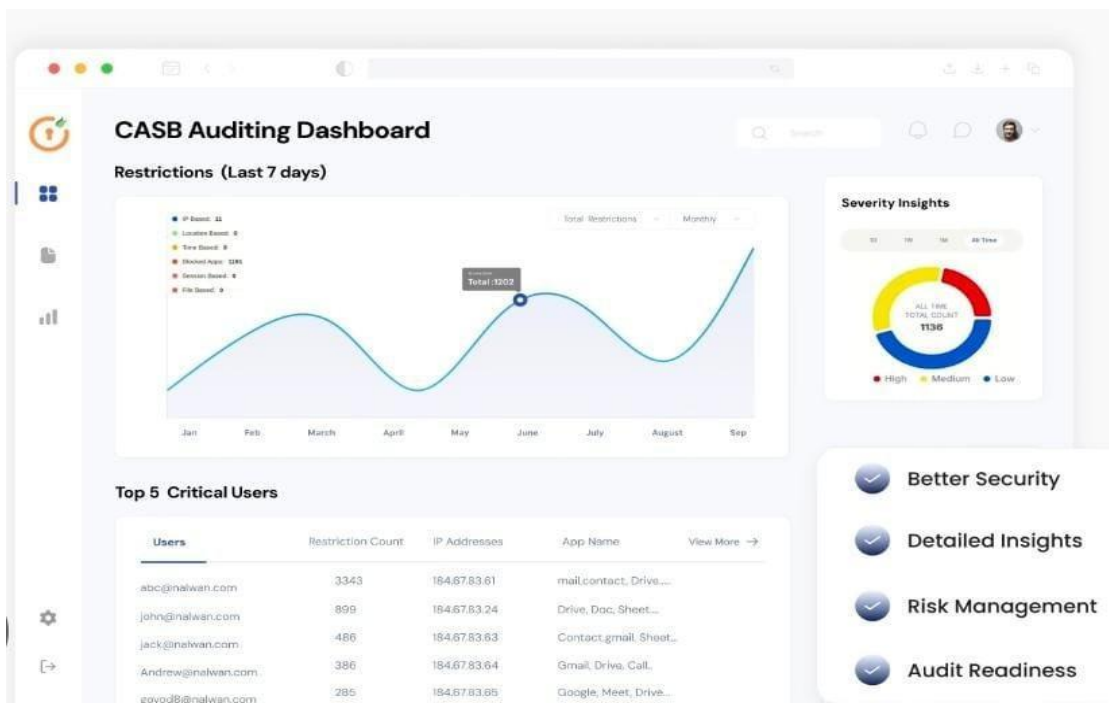
6. Generate Reports:
Use built-in reporting features to export summaries and dash boards.
7. Analysis:
Evaluate how effectively the CASB identified threats and enforced policies.
Note any usability or integration issues.

OUTPUT:

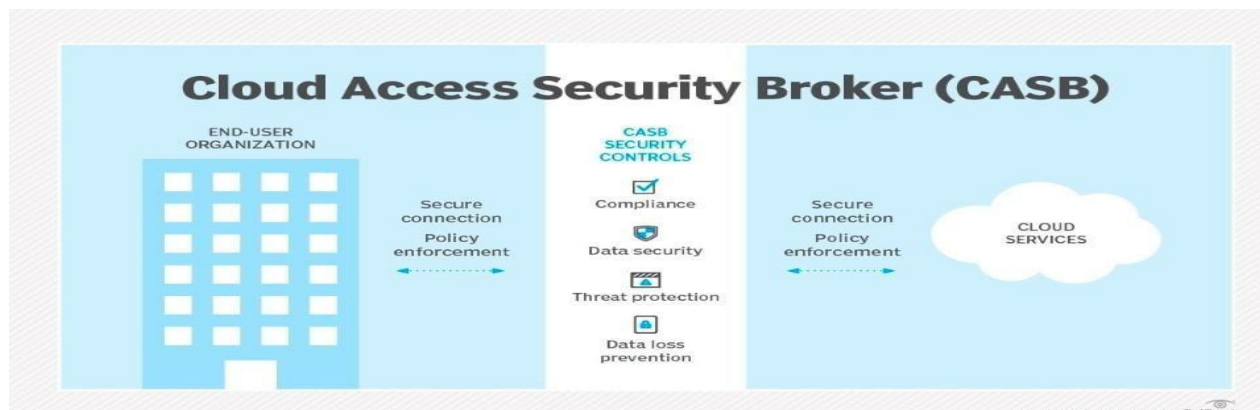
1. CASB dashboard showing cloud app access, upload/download sizes, and user activity insights.



CASB audit dashboard displaying critical user activities, policy violation severities, and audit readiness insights.



2. Shadon IT detection CASB example



RESULT:

CASB successfully logs and monitors cloud app activities and Security policies are enforced correctly (alerts, blocks).

EXP.NO: 11

DATE:

CONFIGURING AND MANAGING WEB APPLICATION FIREWALLS (WAFS) TO PROTECT CLOUD APPLICATIONS FROM COMMON WEB-BASED ATTACKS AND THREATS

AIM:

To configure a Web Application Firewall (WAF) on a cloud-hosted application and test its effectiveness against common web-based attacks and threats.

PREREQUISITES:

- Cloud Platform Account (e.g., AWS, Azure, Google Cloud)
- Cloud-native WAF Service (e.g., AWS WAF, Azure Front Door WAF, Google Cloud Armor)
- Sample Web Application (e.g., a simple vulnerable app like DVWA or a custom-built application)
- Attack Simulation Tools – OWASP ZAP, Burp Suite, Nikto, SQLMap
- Basic CLI Tools (optional) - AWS CLI / Azure CLI / gcloud SDK

THEORY:

A Web Application Firewall (WAF) protects web applications by filtering, monitoring, and blocking malicious HTTP/S traffic. It can detect and mitigate attacks like SQL injection, Cross-Site Scripting (XSS), file inclusion, and session hijacking. Cloud providers offer WAFs that integrate with application load balancers and API gateways, providing customizable security rules and real-time protection.

PROCEDURE:

1. Deploy a Sample Web Application

- Host a simple app on a cloud service (e.g., EC2 on AWS, App Service on Azure, App Engine on GCP).

2. Set Up the WAF

- Navigate to WAF management console (AWS WAF, Azure Front Door WAF, etc.).
- Create a WAF policy.
- Attach the WAF policy to your application endpoint (Load Balancer, CDN, or directly to App).

3. Configure WAF Rules

- Enable Managed Rules (example: AWS Core rule set, OWASP Top 10 protections).
- Add Custom Rules
 1. Block requests with suspicious User-Agents.
 2. Rate limit requests from a single IP exceeding threshold.
 3. Block IPs from restricted geographies.

4. Testing WAF Protection

- Use OWASP ZAP or Burp Suite to launch test attacks (SQLi, XSS).
- Observe if WAF blocks/alerts on malicious payloads.

5. Monitor and Fine-Tune

- Review WAF logs.
- Adjust rule thresholds (reduce false positives/negatives).
- Add exception handling if necessary.

SOURCE CODE:

Creating a WAFv2 Web ACL with AWS CLI:

```
aws wafv2 create-web-acl \  
    --name my-web-acl \  
    --scope REGIONAL \  
    --default-action Block={} \  
    --visibility-config  
SampledRequestsEnabled=true,CloudWatchMetricsEnabled=true,MetricName=my-web-acl \  
    --region us-east-1 \  
    --rules file:///rules.json
```

Sample rules.json:

```
[  
  {  
    "Name": "AWS-AWSManagedRulesCommonRuleSet",  
    "Priority": 1,  
    "Statement": {  
      "ManagedRuleGroupStatement": {  
        "VendorName": "AWS",  
        "Name": "AWSManagedRulesCommonRuleSet"  
      }  
    },  
    "OverrideAction": {  
      "None": {}  
    },  
  },  
]
```

```
"VisibilityConfig": {  
  "SampledRequestsEnabled": true,  
  "CloudWatchMetricsEnabled": true,  
  "MetricName": "CommonRuleSet"  
}  
}  
]
```

RESULT:

The WAF successfully detected and blocked common web-based attacks and threats while allowing legitimate traffic to pass through. This demonstrates the ability of the configured WAF to protect cloud applications against common web-based threats.

EXP.NO: 12

DATE:

ASSESSING THE SECURITY OF SERVERLESS APPLICATIONS AND FUNCTIONS DEPLOYED ON CLOUD PLATFORM FOR VULNERABILITIES AND MISCONFIGURATIONS

AIM:

To evaluate the security posture of serverless applications and cloud functions by identifying potential vulnerabilities and misconfigurations that could be exploited by malicious actors.

PREREQUISITES:

- A cloud service provider (e.g., AWS Lambda, Google Cloud Functions, Azure Functions)
- Serverless Framework / AWS CLI / GCP CLI
- Security tools:
- OWASP ZAP
- CloudSploit
- ScoutSuite
- Snyc

THEORY:

1.Cloud Sploit

Used for scanning misconfigurations in cloud environments. Now part of Aqua Security. Can be used as a command-line tool or via the Aqua Console.

2. Scout Suite

An open-source cloud security auditing tool. It works by querying cloud APIs and analyzing configurations.

To run on AWS:

3. Snyc

Finds known vulnerabilities in dependencies of Node.js, Python, etc.

To scan a project:

4. OWASP ZAP

A powerful web app scanner used for testing exposed endpoints in serverless APIs. You can use the ZAP GUI or run it headless via:

These tools help identify misconfigurations, vulnerable packages, and security flaws in cloud-based serverless deployments.

PROCEDURE:

1. Setup the Environment:
 - Deploy a sample serverless function using AWS Lambda or similar.
 - Trigger it using an API Gateway or equivalent.
2. Intentionally Misconfigure Settings:
 - Assign excessive IAM permissions.
 - Store secrets in plaintext environment variables.
 - Open public access to storage (e.g., S3).
3. Run Security Scans:
 - Use ScoutSuite or CloudSploit to check cloud settings.
 - Scan dependencies with Snyk.
 - Test APIs using OWASP ZAP.
4. Review IAM Roles and Permissions:
 - Examine permissions for unnecessary access.
5. Record Findings and Recommendations:
 - Log all issues and suggest best practices.

RESULT:

The experiment successfully highlighted multiple vulnerabilities and misconfigurations in the cloud-based serverless application. Security auditing tools like ScoutSuite, CloudSploit, Snyk, and OWASP ZAP proved effective in identifying real-world security issues. These findings emphasize the need for secure coding, proper configuration, and regular security audits in serverless environments.

EXP.NO: 13

DATE:

CONFIGURING AND TESTING DDoS PROTECTION MECHANISM TO SAFEGUARD CLOUD APPLICATIONS AND SERVICES FROM DDoS ATTACKS

AIM:

To configure cloud-based DDoS protection and simulate a DDoS attack using a Python script to verify the application's ability to resist such attacks.

PREREQUISITES:

- Cloud Provider Account (AWS, Azure, GCP)
- Sample cloud-hosted web application (publicly accessible)
- DDoS protection enabled (e.g., AWS Shield, Azure DDoS Protection)
- Monitoring tools (CloudWatch, Azure Monitor)
- Python environment for simulation

THEORY:

DDoS protection works by identifying abnormal traffic patterns and automatically mitigating attack vectors such as SYN floods, UDP floods, or HTTP request floods. Cloud providers offer native tools to protect resources from high traffic and preserve availability.

Steps:

1. Deploy the Application:

Launch a VM or cloud web service with a sample app (e.g., Nginx on Ubuntu, or a Node.js app). Ensure it's reachable from a public IP or domain.

2. Configure DDoS Protection:

Enable AWS Shield Standard (default for all AWS services) or Shield Advanced.

Set up security rules via AWS WAF, Azure NSG, or Cloud Armor.

Add rate-limiting and IP filtering rules.

3. Monitor Baseline Performance:

Use AWS CloudWatch or Azure Monitor to log normal CPU, RAM, and traffic usage.

4. Simulate DDoS Attack with Python

Note: This script is for lab testing only. Do not use it against unauthorized or production systems.

SOURCE CODE:

```
import threading
import requests
import time
target_url = "http://<your-cloud-app-public-ip-or-domain>" # Replace with your actual URL
num_threads = 50 # Number of threads to simulate concurrent requests
duration = 30 # Duration of attack in seconds
def attack():
    end_time = time.time() + duration
    while time.time() < end_time:
        try:
            response = requests.get(target_url)
            print(f'Request sent - Status: {response.status_code}')
        except Exception as e:
            print(f'Error: {e}')
threads = []
print(f'Starting simulated DDoS attack on {target_url} for {duration} seconds...')
for i in range(num_threads):
    t = threading.Thread(target=attack)
    t.start()
    threads.append(t)
for t in threads:
    t.join()
print("Simulation completed.")
```

OUTPUT:

Input: edumate.sairamit.edu.in

Output: Request sent - Status: 770915

Starting simulated DDoS attack on <https://edumate.sairamit.edu.in/> for 30 seconds...

Simulation completed

Observe and Log Behavior:

- Check cloud monitoring dashboards
- CPU/network spike
- Latency or errors
- Blocked IPs or throttling by WAF/DDoS tools

RESULT:

The cloud protection mechanism successfully detected and mitigated a surge in traffic, maintaining uptime and preventing server overload.

EXP.NO: 14

DATE:

SIMULATING CLOUD SECURITY INCIDENTS AND PRACTICING INCIDENT RESPONSE PROCEDURES TO EFFECTIVELY DETECT, ANALYZE, AND MITIGATE SECURITY THREATS

AIM:

To simulate cloud security incidents within the Microsoft Azure environment and practice incident response procedures using Azure-native tools to detect, analyze, contain, and mitigate security threats effectively.

ALGORITHM:

1. Setup Cloud Environment
 - Deploy virtual machines, storage accounts, and networks in Azure.
 - Enable Microsoft Defender for Cloud and Azure Monitor.
2. Simulate Security Incidents
 - Trigger controlled incidents such as:
 - Brute force login attempts.
 - Malicious script execution.
 - Unauthorized port access.
2. Detect Incidents
 - Use Microsoft Defender for Cloud, Sentinel, and Azure Security Center to detect anomalies.
 - Monitor alerts from Azure Monitor and Log Analytics.
3. Analyze the Incident
 - Examine security alerts and logs.
 - Use Azure Sentinel's Investigation Graph and Workbooks.
4. Respond to Incident
 - Trigger automated playbooks (Logic Apps).
 - Isolate affected resources (e.g., disable VM NIC, block IP).
 - Apply patches or reconfigure security settings.
5. Mitigate and Recover
 - Perform remediation steps (e.g., rotate credentials, delete malicious files).
 - Conduct post-incident reviews and document lessons learned.
6. Verify and Close
 - Confirm issue resolution.
 - Update security policies to prevent recurrence.
 - Close the incident ticket and generate reports.

THEORY:

Cloud environments are dynamic and susceptible to a wide range of cyber threats. Simulating incidents allows organizations to test their detection and response capabilities in a controlled manner, ensuring preparedness for real-world attacks.

Key Concepts:

- Microsoft Defender for Cloud: Provides threat detection and protection for Azure resources.
- Azure Sentinel: A cloud-native SIEM/SOAR platform for alert correlation, investigation, and automated response.
- Log Analytics & Azure Monitor: Used for collecting and analyzing telemetry data and logs.
- Incident Response Lifecycle:
 1. Preparation
 2. Detection & Analysis
 3. Containment, Eradication & Recovery
 4. Post-Incident Review

PROCEDURE:

Step 1: Setup Environment

- Deploy a VM (e.g., Windows Server or Linux).
- Install logging agents (Azure Monitor Agent).
- Enable Defender for Cloud and Azure Sentinel.
- Connect VM to Sentinel using a Log Analytics Workspace.

Step 2: Simulate Incidents

- Example 1: Attempt multiple failed logins using brute-force script.
- Example 2: Upload and execute a suspicious PowerShell script.
- Example 3: Open RDP port (3389) to the internet.

Step 3: Detect the Incidents

- Open Defender for Cloud > Alerts to view triggered alerts.
- Use Azure Sentinel > Incidents to view correlated incidents.

Step 4: Analyze the Threat

- Review the Investigation Graph in Sentinel.
- Analyze related alerts, entities (users, IPs, devices), and timeline.

Step 5: Respond to Incident

- Trigger a playbook to:
 - Send alert email to security team.
 - Disable VM NIC.
 - Block suspicious IP addresses.

Step 6: Mitigation

- Run a malware scan on the VM.
- Remove the malicious file/script.
- Change VM password and enable MFA.

Step 7: Post-Incident

- Review logs.
- Update firewall rules, NSG, and policies.
- Write an incident report and lessons learned.

OUTPUT:

Example: Defender for Cloud Alert

Severity	Alert Title	Resource	Description
High	Brute Force Attack Detected on VM	vm-simulation-test	Multiple failed login attempts detected
Medium	Suspicious Script Execution Detected	vm-simulation-test	PowerShell with obfuscation executed

Example: Sentinel Investigation

- **Incident:** Suspicious login behavior detected.
- **Entities:** User: attacker@domain.com, IP: 203.0.113.10
- **Timeline:**
 - 10:00 AM: Failed login x20
 - 10:02 AM: Successful login
 - 10:05 AM: PowerShell script executed
- **Automated Response:** VM NIC disabled, alert email sent.

Example: Final Report

Step	Status	Notes
Detection	Successful	Detected via Defender Alert
Containment	Completed	NIC disabled, port blocked
Remediation	Completed	Script deleted, password changed
Post-Incident	Completed	Policy updated, RDP restricted

RESULT:

Thus the Experiment is successfully completed and the output is verified.

EXP.NO: 15

DATE:

AUTOMATING COMPLIANCE CHECKS AND AUDITING CLOUD RESOURCES AGAINST REGULATORY STANDARDS AND INDUSTRY BEST PRACTICES USING CLOUD-NATIVE COMPLIANCE FRAMEWORKS

AIM:

To implement automated compliance checks and audit cloud infrastructure using native tools provided by cloud providers, ensuring adherence to regulatory standards and industry best practices.

PREREQUISITES:

- Basic understanding of cloud computing
- AWS/Azure/GCP account with admin privileges
- Familiarity with regulatory standards (e.g., CIS, NIST)
- CLI and cloud console navigation skills

TOOLS REQUIRED:

- AWS Config / Security Hub (for AWS)
- OR Azure Policy / Security Center (for Azure)
- OR GCP Security Command Center
- Cloud CLI (AWS CLI / Azure CLI / gcloud)
- Web browser
- Text editor

PROCEDURE:

Step 1: Setup AWS Config

1. Login to AWS Console.
2. Navigate to AWS Config > Click Get Started.
3. Select resources to record (e.g., EC2, S3).
4. Choose an S3 bucket to store configuration history.
5. Set up Amazon SNS topic for notifications.

Step 2: Enable Security Hub

1. Go to AWS Security Hub.
2. Enable the service and integrate it with AWS Config.
3. Enable standards like CIS AWS Foundations Benchmark.

Step 3: Compliance Check

1. Navigate to Security Hub > Findings.
2. View findings based on the enabled standards.

3. Identify resources that are non-compliant.

Step 4: Auto-remediation (Optional)

1. Create a Lambda function to fix non-compliance (e.g., block public access on S3).
2. Trigger function using AWS Config Rules or EventBridge.

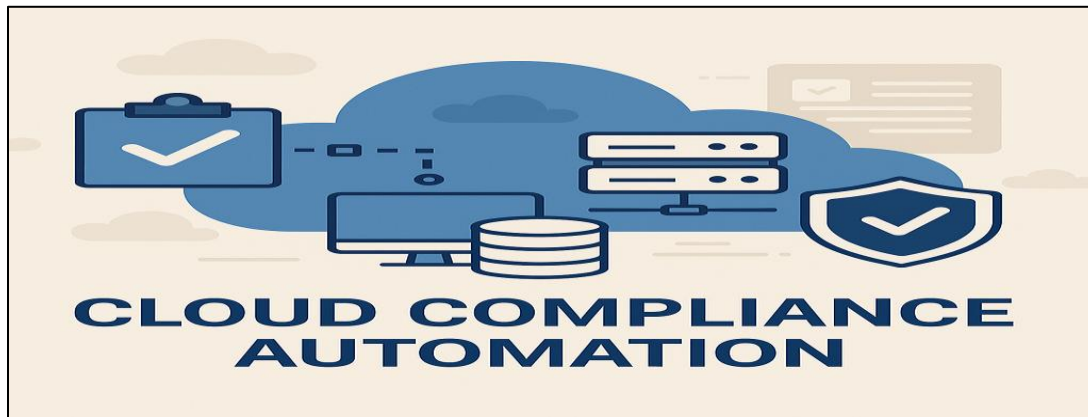
Step 5: Generate Compliance Report

1. Go to AWS Config > Compliance.
2. Export compliance report for audit use.



OBSERVATION TABLE:

Resource Type	Compliance Rule	Compliant(Yes/No)	Remediation Applied (Yes/No)
S3 Bucket	Public access blocked	No	Yes
EC2	Approved instance type	Yes	-
IAM	MFA enabled for all users		



RESULT:

Successfully implemented AWS-native compliance checks. Identified and reported misconfigured resources. Demonstrated the ability to audit and export compliance reports.